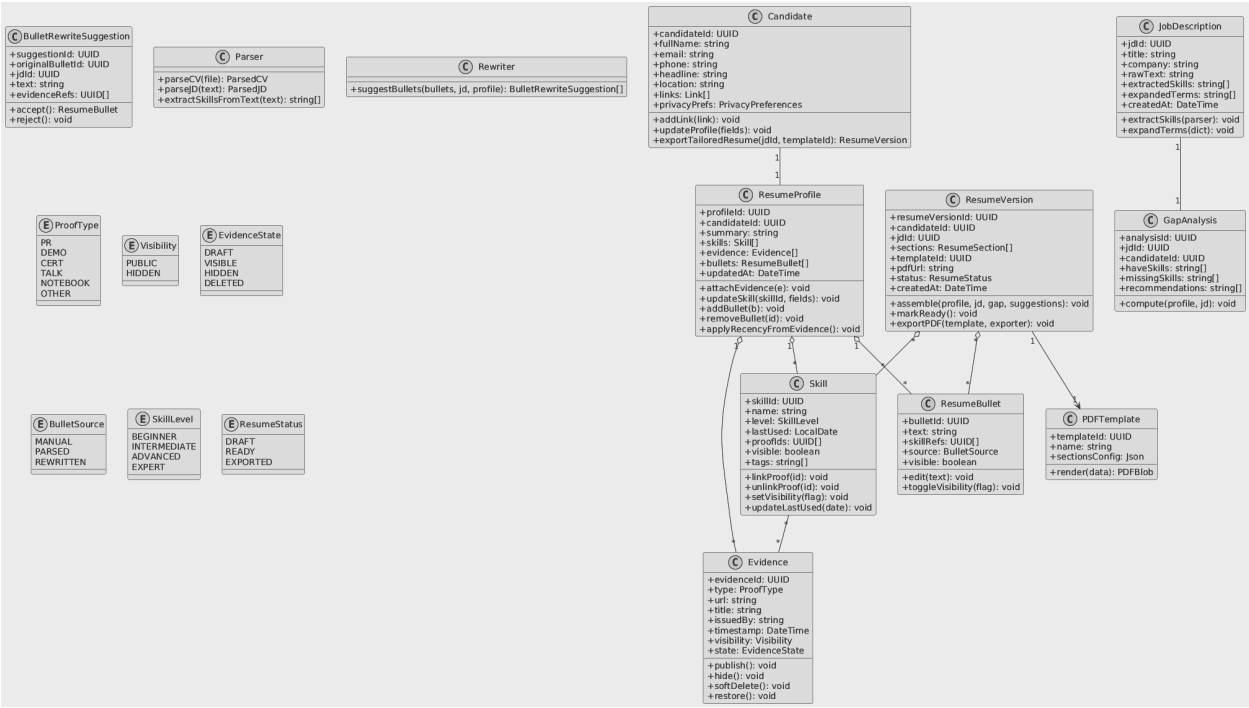
CSE470: Software Engineering

Project Purpose

Our goal is to build a platform that builds a living, evidence backed Resume that auto updates from real life activity of a candidate and recruiters use the platform to screen candidates that match the job descriptions perfectly.

Class Diagram:



Functional requirements

<u>Module 1</u>: Profile and Account Management

<u>Features</u>:

1. Authentication and Roles
    - The system will allow users (Candidate / Recruiter) to register with Name, Email, Password.
    - Upon successful registration the system will send an email verification (via dev-mode Nodemailer).
    - The system will prevent accessing any dashboard if they are logged out route guards for dashboards.
    - Users can log out.
2. Profile
    - A candidate will be able to update their name, email, photo, change password.
    - A recruiter will be able to update their name, org details, and change password.
    - The system will ask for two factor authentication when changing passwords.
3. Shared services
    - Mail service used by: email verification + (later) weekly recap send.
    - New user registers will receive dev email links and will see the correct dashboard.
    - Unauthorized routes are blocked.

<u>Module 2</u>: Resume & Skill Management

Features:

1. CV Upload & Parse
    - The system will provide a local stub parser that uses regular expressions and keyword matching to extract skills and bullet points from the Resume.
    - The system will add an optional switch to enable AI assistance (Module 4).
    - It will store a structured model (skills, bullets, proofs, placeholders).
2. Skill Matrix
    - The user will be able to add/edit/delete skills.
    - They can update “last used. For any skill”
    - Add bullet points under each skill.
3. Proofs
    - The user will be able to add links (GitHub PRs, certs, demos).
    - The system will add ‘Visibility toggle: Public / Private’.
    - The system will provide a Proof gallery page where users can view and manage their proofs.
4. Job Description-Tailored PDF
    - The system will allow users to Paste JD (text).
    - It will highlight matched skills in the users profile that match the job description (from local comparison first).
    - It will choose a template, show preview and then download PDF.

- Upload CV → skills appear → edit + add proofs → paste JD → export clean PDF.
- PDF shows only Public proofs.

Module 3: JD (Job Description) Parsing & Matching (Recruiter)

Features:

1. JD Input
    - The system will allow a recruiter to input a JD by extracting keywords & tools.
    - The system will use a local dictionary for synonyms (e.g., React → RTK).
    - It will also provide an optional switch to AI module /parse-jd.

2. Candidate Match (two modes)
    - In Sprint 1–2: the system will generate a matching score for candidates on skill overlap, recently used skills and number of public proofs
    - In Sprint 3–4: optional switch to Module 4's /match.
    - Show match %, missing skills, recently used badges.
    - Public-only proofs visible.
3. Shortlist
    - The recruiters will be able to add/remove/tag (Interview / Maybe / Hold).

- The system will persist the recruiters shortlist so it is maintained across browsers (local storage or DB).

4. Acceptance:
    - Once a JD is posted, the system will display a ranked list of candidates.
    - Each row will show “why matched” notes.
    - Bias-reduced view hides name/photo/school.
    - Shortlist persists across refresh.

<u>Module 4</u>: AI Service (CV/JD Parsing, Match, Rewrite)

<u>Features</u>:

1. CV Parsing API
    - The API endpoint will return a structured list of skills found in the input CV and the source text position for each skill.

2. JD Parsing API
    - The api endpoint will return a list of skills and expanded terms in the JD

3. Match API
    - It will return matching scores and why they are in match.

4. Rewrite API + Tester
    - It will return 3 bullet suggestions, grounded in input.

- One-page tester to call all endpoints.
- Median latency will be < 900 ms on fixtures.
- Reasons include source spans
- Rewrites never add facts not present in the inputs.

.

Nonfunctional requirements

4.1 Performance Requirements

Fast page loading

The system should load all major pages (Dashboard, Skill Matrix, JD Match page) within 3–5 seconds on a normal internet connection.

Quick matching response

When a recruiter runs the JD–Candidate matching feature, the match result should be generated within 3 seconds for up to 50 candidates.

Efficient Resume Parsing

When a candidate uploads a Resume, the system should extract skills, experience, and tools within 5 seconds.

Smooth navigation

Moving between pages such as Profile, Skills, Proofs, JD Match, and Recruiter View should be smooth, without freezing or crashing.

4.2 Usability Requirements

Simple and clean UI

The interface should be easy to understand for new users, with clear buttons such as "Upload CV", "Add Skill", "Match with JD", and "View Proof".

Mobile-friendly design

The system should work properly on both desktop and mobile devices (responsive layout).

Clear error messages

If the user enters invalid input (e.g., wrong file type), the system should show a clear and helpful error message explaining what needs to be fixed.

Guided actions

Important steps like uploading a Resume, pasting a JD, generating a PDF, or sending the weekly summary should include simple instructions and visible buttons.

4.3 Security Requirements

Secure password storage

All user passwords must be stored in encrypted (hashed) form within the database.

Protected routes

Candidate and recruiter dashboards must only be accessible after login. Direct access to these pages should be blocked.

Role-based access control

Candidates cannot access recruiter-only features, and recruiters cannot access or edit candidate private data such as bullets or password settings.

Safe proof links

The system should validate proof links and text inputs to prevent harmful scripts or malicious links from being added.

4.4 Reliability & Availability Requirements

Data consistency

All saved data (skills, bullets, proofs, match results) must remain stored correctly even after refreshing or logging out.

Graceful failure

If any AI service (CV parsing, JD parsing, matching, rewriting) is temporarily unavailable, the system should show a friendly message instead of crashing.

Basic backup

The database should have regular backups (e.g., daily during development) to prevent data loss.

4.5 Maintainability Requirements

Modular code structure

The project should be built using separate modules (Login, Resume, Skills, JD Parsing, Recruiter Panel, AI Services) to make development and updates easier.

Clear API separation

The frontend and backend should communicate using clear API endpoints (such as /parse-cv, /parse-jd, /match, /rewrite) so changes in one part do not break others.

Commented code

Important logic like Resume parsing, matching, skill updates, and weekly summaries should contain clear comments for future developers.

4.6 Scalability & Extensibility Requirements

Support more users in future

The system should be built in a way that allows more candidates and recruiters to use it in the future without major code changes.

Easy to extend with new features

New features such as more Resume templates, additional proof types, or new AI tools should be easy to add without rewriting the full project.

4.7 Privacy Requirements

Control over proof visibility

Candidates should have full control over marking proof links as Public or Private. Private proofs must not appear in recru**iter views.**

Use of data only inside system

User data (Resume, skills, proofs) must only be used inside the ProofDIn system for matching and profile display, not shared with any external services.